



Event Management & Ticket Booking System



ZennialPro

(By: Vijay Sai Nunna – ZPVN-10116)



1.Overview

The Event & Ticket Booking System is a **web-based backend application** built using **FastAPI** and **MongoDB (async with Motor)**. It automates the process of **event creation, venue approval, and ticket booking**, ensuring **real-time seat tracking, security, and scalability**.

The system provides role-based access:

- **Admins** can register, log in, create/manage events, approve venues, and oversee user activities.
- **Users** can register, log in, browse events, book tickets, and manage their bookings.

All operations are validated using JWT authentication, password hashing (Passlib), Pydantic validation, and centralized logging/exception handling.

1.1 Problem Statement

Managing events and ticket booking manually is inefficient, error-prone, and not scalable.

Existing systems often lack:

- Secure user authentication,
- Real-time seat availability validation,
- Centralized management of events/venues/bookings,
- Ability to handle multiple concurrent users.

Hence, there is a need for a secure, automated, and scalable digital solution that simplifies the process for both event organizers and participants..

1.2 Objectives

The key objectives of the Event & Ticket Booking System are:

- Provide a secure user and admin management system with authentication.
- Enable admins to manage events and venues digitally.

- Allow users to browse events and book/cancel tickets seamlessly.
- Ensure real-time seat tracking to prevent overbooking.
- Maintain a modular and scalable backend ready for future enhancements.
- Implement logging and exception handling for reliability.

1.3 Scope

The scope of the system includes backend functionality for user management, event/venue management, and ticket booking.

Features Covered:

- User registration, login, and role-based access.
- Event creation, modification, and deletion.
- Venue addition and approval by admins.
- Ticket booking and cancellation with seat rollback.
- MongoDB-based data storage for scalability.
- Centralized logging and error handling.

Future Scope:

- Online payment integration.
- Multi-location event and ticket categories.
- Real-time booking notifications.
- Mobile app interface.
- Analytics dashboards for admin

2. System Architecture

2.1 High-Level Architecture

The Event & Ticket Booking System follows a client–server model:

- **Client:** Frontend app or Postman for testing APIs.
- **Backend (FastAPI):** Handles requests, authentication, booking logic.
- **Database (MongoDB):** Stores users, events, venues, and bookings.

Flow Example:

- A user logs in and requests to book tickets.
- FastAPI validates the token and checks seat availability.
- MongoDB stores the booking details and updates availability.
- The system returns a confirmation or error response.

2.2 Components Overview

2.2.1 Authentication & Authorization Module:

- Handles registration, login, and JWT-based token generation.
- Provides role-based access (admin/user).

2.2.2 User Management Module

- Profile management (view, update, delete).
- Secure password storage with hashing.

2.2.3 Admin Management Module

- Event creation, update, and deletion.
- Venue approval and monitoring.

2.2.4 Event & Venue Module

- Store event/venue details (capacity, type, city, etc.).
- Link venues to events.

2.2.5 Booking Module

- Ticket booking with seat validation.
- Cancellation with seat rollback.
- Booking history for users.

2.2.6 Database Layer (MongoDB)

- Collections: Users, Events, Venues, Bookings.
- Indexing for performance and scalability.

2.2.7 Logging & Error Handling

- Tracks system activity and errors for transparency.

ZennialPro

3.Features

1 .Secure login and authentication with JWT:

- Uses JWT (JSON Web Tokens) to securely authenticate users and admins.
- Prevents unauthorized access by validating tokens on every request.
- Supports stateless authentication, reducing server overhead.
- Protects sensitive information with password hashing (Passlib)Role-based access (Admin/User)

2.Role-based access (Admin/User):

- Clearly separates admin privileges from user privileges.
- Admins can create, approve, and manage events/venues.
- Users can browse events, book tickets, and cancel bookings.
- Ensures security by restricting critical operations to admins only.

3.Real-time seat tracking:

- Tracks availability of seats in real time to prevent overbooking.
- Updates seat counts instantly after new bookings or cancellations.
- Ensures fair access to tickets during high-demand events.
- Maintains accurate capacity information for venues and events.

4.Ticket booking and cancellation:

- Users can book tickets seamlessly with unique booking IDs.
- Cancellations roll back seats, making them available for others.
- Booking records are securely stored in MongoDB for reference.
- Supports easy viewing and management of past bookings.

5.Event and venue CRUD operations:

- Admins can Create, Read, Update, and Delete events and venues.
- Stores essential details like location, type, capacity, and date.
- Venues require admin approval before being linked to events.
- Helps organizers maintain structured and up-to-date information.

6.Centralized MongoDB data storage:

- Stores all user, event, venue, and booking data in a single database.
- Uses async operations with Motor for high concurrency.
- Supports indexing and querying for fast data retrieval.
- Ensures scalability to handle large events and many users.

7.Logging and exception handling:

- Centralized logs track all user and admin activities.
- Helps in monitoring and debugging errors effectively.
- Improves system reliability and maintainability.
- Provides error messages that guide users when issues occur.

8.Modular, asynchronous backend design:

- Backend is divided into routers, services, models, and utilities.
- Follows clean architecture for scalability and easy updates
- sync FastAPI design allows multiple requests simultaneously.
- Makes the system robust, maintainable, and future-ready.

3.1Technology Stack & State

- **Language:** Python 3.x
- **Framework:** FastAPI
- **Database:** MongoDB (Motor async driver)
- **Authentication:** JWT, Passlib
- **Validation:** Pydantic models
- **Logging:** Centralized Python logging
- **Server Engine:** Uvicorn (ASGI server)

4.API Design & Functional Modules

a. User Module

- **Features:** Registration, Login (JWT), Profile Management, Logout
- **Endpoints:**
 - POST /users/register – Register a new user
 - POST /users/login – User login (JWT issued)
 - GET /users/me – Get current user info
 - PUT /users/me– Update user profile
 - DELETE /users/me – Delete user account

b. Admin Module

- **Features:** Admin Authentication, Manage Users, Manage Admins
- **Endpoints:**
 - POST /admins/register – Register an admin
 - POST /admins/login – Admin login
 - GET /admins/me – Get current admin info
 - GET /admins – Get all admins

- GET /users – Get all users (admin only)
- PUT /admins/{id} – Update admin
- DELETE /admins/{id} – Delete admin

c. Event Module

- **Features:** Event CRUD, Event Retrieval by type/id

- **Endpoints:**

- POST /events – Create event (admin only)
- PUT /events/{id} – Update event (admin only)
- DELETE /events/{id} – Delete event (admin only)
- GET /events/{id} – Get event by ID (public)
- GET /events/type/{type} – Get events by type
- GET /events – Get all events

d. Venue Module

- **Features:** Venue CRUD, Venue Retrieval

- **Endpoints:**

- POST /venues – Create venue (admin only)
- PUT /venues/{id} – Update venue (admin only)
- DELETE /venues/{id} – Delete venue (admin only)
- GET /venues/{id} – Get venue by ID
- GET /venues/type/{type} – Get venues by type
- GET /venues – Get all venues

e. Booking Module

- **Features:** Seat Booking, Approval/Rejection, Cancel Booking

- **Endpoints:**

- POST /bookings – Book event seats (user)
- GET /bookings/me – Get user's bookings (user)
- PUT /bookings/{id}/cancel – Cancel booking (user)
- PUT /bookings/{id}/approve – Approve booking (admin only)
- PUT /bookings/{id}/reject – Reject booking (admin only)
- GET /bookings – Get all bookings (admin only)

5.Database Design

The backend uses **MongoDB** as the primary database with the **Motor async driver** for non-blocking I/O operations.

Data is organized into **collections** representing the main entities of the system. Relationships are handled through document references (IDs) and embedding when necessary.

a. Collections & Schema Design

1. Users Collection

- **Purpose:** Stores all registered users and their profile details.
- **Fields:**
 - **user_id (string, unique)** – Unique user identifier
 - **username (string, unique)** – User's full name or username
 - **email (string, unique)** – User's email address
 - **password (hashed string)** – Securely hashed password
 - **role (string)** – Either user or admin
- **Notes:**
 - Passwords are stored in hashed form using secure algorithms (e.g., bcrypt).
 - Role-based access control is enforced via the `role` field.

2. Admins Collection

- **Purpose:** Stores admin accounts separately for easy management.
- **Fields:**
 - **admin_id (string, unique)** – Unique admin identifier
 - **username (string, unique)** – Admin name
 - **email (string, unique)** – Admin email
 - **password (hashed string)** – Securely hashed password
 - **role (string)** – Always admin
- **Notes:**
 - Admins can manage users, events, venues, and bookings.

3. Venues Collection

- **Purpose:** Maintains venue details where events are conducted.
- **Fields:**
 - **venue_id (string, unique)** – Unique venue identifier
 - **name (string)** – Venue name
 - **city (string)** – City of venue
 - **location (string)** – Address / landmark

- **capacity (integer)** – Maximum seating capacity
 - **type (string)** – Type of venue (e.g., tech, music, gym)
 - **events (array of strings)** – List of event names conducted here
- **Notes:**
 - One venue can host multiple events.

4.Events Collection

- **Purpose:** Stores information about all created events.
- **Fields:**
 - **event_id (string, unique)** – Unique event identifier
 - **title (string)** – Event name
 - **type (string)** – Event category (e.g., music, tech, sports)
 - **venue_id (string, reference)** – Linked venue ID
 - **total_seats (integer)** – Maximum seats available
 - **booked_seats (array of integers)** – List of occupied seat numbers
 - **available_seats (integer)** – Computed: total_seats - len(booked_seats)
 - **city (string)** – Event city
 - **is_approved (boolean)** – Whether admin approved the event
 - **created_at (datetime)** – Timestamp of creation
- **Notes:**
 - Seat allocation is managed dynamically during bookings.

5.Bookings Collection

- **Purpose:** Tracks ticket bookings made by users.
- **Fields:**
 - **booking_id (string, unique)** – Unique booking reference
 - **user_id (string, reference)** – ID of the user who booked
 - **user_name (string)** – Name of the user
 - **event_id (string, reference)** – ID of booked event
 - **seats_booked (array of integers)** – List of booked seat numbers
 - **status (string)** – pending, approved, rejected, or cancelled
 - **booking_date (datetime)** – Date of booking
 - **event (embedded object)** – Snapshot of event details
 - **venue (embedded object)** – Snapshot of venue details
- **Notes:**
 - Status changes update seat availability in Events collection.
 - Rejected/Cancelled bookings roll back seats.

6.Request & Response Examples

a.User Modules

1.Register

POST /auth/register

```
{
  "Full_name": "Vijay sai",
  "email": "Vijay@email.com",
  "phone": "8463736738",
  "password": "Vijay@123",
  "confirm_password": "Vijay@123"
```

Response:

```
_id: ObjectId('68bf0345b50faf4558520944')
user_id: "USR004"
Full_name: "Vijay Sai Nunna"
email: "VJ@email.com"
phone: "9652394215"
password: "$2b$12$B0GqvUF1sRPfNXtNNOHAJO56Kdsh42Jow9eqgE75F1fM3.YVa0bzi"
is_active: true
created_at: 2025-09-08T16:24:37.853+00:00
```

2. Login-User

POST /auth/login

```
{
  "email": "Vijay@example.com", "password": "sECURED123"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

3.Get User Info

GET /users/me

Response:

```
_id: ObjectId('68bf0345b50faf4558520944')
user_id: "USR004"
Full_name: "Vijay Sai Nunna"
email: "VJ@email.com"
phone: "9652394215"
password: "$2b$12$B0GqvUF1sRPfNXtNNOHAJO56Kdsh42Jow9eqgE75F1fM3.YVa0bzi"
is_active: true
created_at: 2025-09-08T16:24:37.853+00:00
```

4.Update User Info

PUT /users/me/update

```
{
  "name": "Vijay kumar",
  "email": "Vijay.new@example.com"}
```

Response:

```
{
  "message": "Profile updated successfully"
}
```

5.Delete User profile

DELETE /users/me/delete

Response:

```
{
  "message": "User deleted successfully"
}
```

b. Admin Module

1 Register Admin

Endpoint: POST /admins/register

Request

```
{
  "username": "admin1",
  "email": "admin@example.com",
  "password": "AdminPass123",
  "full_name": "System Admin"
}
```

Response

```
{
  "admin_id": "ADM001",
  "username": "admin1",
  "email": "admin@example.com",
  "role": "admin",
  "created_at": "2025-09-10T09:30:22.123Z"
}
```

2. Login Admin

Endpoint: POST /admins/login

Request

```
{
  "email": "admin@example.com",
  "password": "AdminPass123"
}
```

Response

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

3. Get All Users (Admin only)

Endpoint: GET /admins/users

Response

```
[
  {
    "user_id": "USR001",
    "username": "kiran",
    "email": "kiran@example.com",
    "role": "user"
  },
  {
    "user_id": "USR002",
    "username": "gowri",
    "email": "gowri@example.com",
    "role": "user"
  }
]
```

4. Get All Admins

Endpoint: GET /admins/

Response

```
[
  {
    "admin_id": "ADM001",
    "username": "admin1",
    "email": "admin@example.com",
    "role": "admin"
  }
]
```

c. Events Module

1. Create Event (Admin only)

Endpoint: POST /events/

Request

```
{
  "event_id": "EVT010",
  "title": "Health & Wellness Expo",
  "type": "lifestyle",
  "venue_id": "VEN010",
  "total_seats": 400,
  "city": "Delhi"
}
```

Response

```
{
  "event_id": "EVT010",
  "title": "Health & Wellness Expo",
  "venue_id": "VEN010",
  "total_seats": 400,
  "booked_seats": [],
  "city": "Delhi",
  "is_approved": false,
  "available_seats": 400
}
```

2. Get Event by ID

Endpoint: GET /events/{event_id}

Response

```
{
  "event_id": "EVT010",
  "title": "Health & Wellness Expo",
  "venue_id": "VEN010",
  "total_seats": 400,
  "booked_seats": [1, 2, 3],
  "available_seats": 397,
  "city": "Delhi",
  "is_approved": true
}
```

3. Get All Events

Endpoint: GET /events/

Response

```
[
  {
    "event_id": "EVT001",
    "title": "Python Bootcamp",
    "venue_id": "VEN001",
    "total_seats": 100,
    "available_seats": 97,
    "city": "Hyderabad"
  },
  {
    "event_id": "EVT002",
    "title": "AI Workshop",
    "venue_id": "VEN002",
    "total_seats": 80,
    "available_seats": 78,
    "city": "Hyderabad"
  }
]
```

d. Venues Module

1 Create Venue (Admin only)

Endpoint: POST /venues/

Request

```
{
  "venue_id": "VEN010",
  "name": "Wellness Convention Center",
  "city": "Delhi",
  "location": "Dwarka",
  "capacity": 600,
  "type": "lifestyle"
}
```

Response

```
{
  "venue_id": "VEN010",
  "name": "Wellness Convention Center",
  "city": "Delhi",
  "location": "Dwarka",
  "capacity": 600,
  "type": "lifestyle",
  "events": []
}
```

```
}
```

2 Get Venue by ID

Endpoint: GET /venues/{venue_id}

Response

```
{
  "venue_id": "VEN010",
  "name": "Wellness Convention
Center",
  "city": "Delhi",
  "location": "Dwarka",
  "capacity": 600,
  "type": "lifestyle",
  "events": ["Health &
WellnessExpo"]
}
```

e. Bookings Module

1 Book Tickets (User only)

Endpoint: POST /bookings/

Request

```
{
  "event_id": "EVT010",
  "seats_booked": [1, 2, 3]
}
```

Response

```
{
  "booking_id": "BKG001",
  "status": "pending",
  "seats_booked": [1, 2, 3],
  "event": {
    "event_id": "EVT010",
    "available_seats": 397,
    "booked_seats": [1, 2, 3]
  }
}
```

2 Approve Booking (Admin only)

Endpoint:

PUT/bookings/approve/{booking_id}

Response

```
{
  "booking_id": "BKG001",
  "status": "approved",
  "event": {
    "event_id": "EVT010",
    "booked_seats": [1, 2, 3],
    "available_seats": 397
  }
}
```

3 Reject Booking (Admin only)

Endpoint: PUT /bookings/reject/{booking_id}

Response

```
{
  "booking_id": "BKG001",
```

```
"status": "rejected",
  "event": {
    "event_id": "EVT010",
    "booked_seats": [],
    "available_seats": 400
  }
}
```

4 Cancel Booking (User only)

Endpoint: PUT /bookings/cancel/{booking_id}

Response

```
{
  "booking_id": "BKG002",
  "status": "cancelled",
  "event": {
    "event_id": "EVT010",
    "booked_seats": [],
    "available_seats": 400
  }
}
```



ZennialPro

7.Error Handling

The backend system follows consistent error handling practices to ensure clarity and predictability for API consumers. Each error response contains an HTTP status code, a descriptive message, and (where relevant) additional details.

Error Type	Status Code	Example Response
Unauthorized / Invalid Token	401	{ "detail": "Invalid or expired token" }
Resource Not Found	404	{ "detail": "User / Event / Venue / Booking not found" }
Bad Request / Validation Error	400	{ "detail": "Required field missing or invalid data" }
Conflict / Already Exists	409	{ "detail": "Seat already booked / Email already registered" }
Internal Server Error	500	{ "detail": "An unexpected error occurred" }
Password Errors	400	{ "detail": "Old password incorrect / Reset token invalid or expired" }

Notes:

- All error responses follow a standard JSON format.
- Status codes clearly indicate the type of failure.
- Errors are designed to communicate clearly with clients while hiding sensitive backend details

8. Conclusion & Future Enhancements

8.1 Conclusion

The **Event & Ticket Booking System** provides a **secure, automated, and scalable** backend solution for event management and ticket reservations. By integrating role-based access control, it ensures that users and admins have tailored functionalities.

The system eliminates the inefficiencies of traditional manual booking processes by introducing:

- **Real-time seat tracking** to prevent double booking.
- **Centralized MongoDB data storage** for consistency and scalability.
- **JWT-based authentication** to safeguard sensitive user information.
- **Asynchronous FastAPI architecture**, making the platform capable of handling high traffic with minimal latency.

Overall, it enhances the user experience for attendees, while simplifying administrative tasks for event organizers. The project also demonstrates strong backend development practices like modular design, error handling, and logging.

8.2 Future Enhancements

To make the system more feature-rich and production-ready, the following enhancements can be added:

1. **Payment Gateway Integration** 
 - Enable secure online payments for ticket purchases.
 - Provide support for multiple payment methods (UPI, cards, wallets).
2. **Real-Time Notifications** 
 - Email/SMS/app notifications for booking confirmations, cancellations, or event updates.
 - Push notifications for last-minute event changes.
3. **Mobile App Interface** 
 - Extend the system to Android/iOS apps for accessibility on the go.
 - Synchronize bookings across devices for seamless user experience.
4. **Multi-Location & Event Categories** 
 - Support events happening across multiple venues or cities.
 - Allow category-wise event browsing (music, tech, sports, workshops).
5. **Admin Dashboards & Analytics** 
 - Provide visual insights into bookings, cancellations, revenue, and attendance.
 - Enable data-driven decision-making for event organizers.